Introduction

Stone Game VIII looks, at first glance, like just another two-player game problem — the kind LeetCode has a dozen variations of. But it's a favorite in interview prep circles for a specific reason: it punishes the instinct to brute-force a game tree, and it rewards the ability to notice when a problem's *real* state space is much smaller than it appears.

That gap — between "what the problem statement makes you think you need to track" and "what you actually need to track" — is exactly what interviewers are testing when they hand you a game-theory DP problem. If you can spot that every move here collapses into a single running number, you demonstrate the kind of algorithmic intuition that separates a candidate who memorized patterns from one who understands them. That's the skill this problem builds, and it's the skill worth carrying into every other DP question you'll face.

Problem Overview

You're given a row of stones, each with an integer value (positive or negative). Alice and Bob take turns, Alice first. On each turn, as long as more than one stone remains, the current player must:

1. Take **at least two stones** from the front of the row.
2. Sum their values — that sum becomes the player's score for the turn.
3. Replace those stones with a single new stone holding that sum, placed back at the front.

Play continues until exactly one stone is left. Alice wants to maximize the final difference `(Alice's total score) - (Bob's total score)`. Bob wants to minimize that same difference. Both play optimally. Return the final difference.

Example

Take the array `[-1, 2, -3, 4, -5]`.

- Alice takes the first four stones: `-1 + 2 + (-3) + 4 = 2`. She scores `2`. The row becomes `[2, -5]`.
- Bob has no choice but to take both remaining stones: `2 + (-5) = -3`. He scores `-3`. One stone remains.
- Final answer: `2 - (-3) = 5`.

Notice Bob wasn't trying to lose — he simply ran out of options once the row shrank to two stones. That's the trap in these problems: sometimes "optimal play" looks identical to "no play." The interesting decisions happen earlier, when a player has real freedom over *where* to cut.

Intuition

The instinct most people reach for first is: "track every possible sequence of cuts Alice and Bob could make." That's a mistake, and it's worth understanding *why* before looking at any code.

Start by asking what actually determines the outcome of the rest of the game at any point. It isn't the full history of who took what. It's just two things: **how many stones are left**, and **whose turn it is**. Everything before that point is sunk — it already went into someone's score and can't be un-scored. So instead of tracking sequences, you can track a single number: the current index into the array, i.e., "the game continues from here."

Now think about what a player's *score* actually is at any cut point. Since the stones removed are always a contiguous prefix (you always take from the front), a player's score for a move ending at index `i` is just the sum of the array from the start up through `i`. That's a prefix sum — nothing more exotic.

Here's the key reframe: forget who explicitly grabbed which stones. All that matters, at any point in the game, is the **running gap** between the two players' totals, assuming both play optimally from there onward. Once you frame the problem as "what's the best achievable gap from index `i` onward," it stops being a simulation problem and becomes a classic optimal-substructure problem — the exact shape dynamic programming is built for.

Brute Force Solution

Idea: Recursively try every valid cut point `x` for the current player, recurse into the remainder of the array, and combine results according to whether the current player is maximizing (Alice) or minimizing (Bob).

Algorithm: 1. At each recursive call, given a starting index, try every possible endpoint `x` for the current prefix. 2. For each choice, compute the prefix sum, then recurse on the remaining suffix with the turn flipped. 3. Alice's calls take the max over all choices; Bob's take the min. 4. Base case: one stone left, game over.

Advantages: Directly mirrors the problem statement — easy to reason about correctness.

Disadvantages: At every index you can branch into every remaining split, so the number of game states explodes combinatorially. For an array of length `n`, this is **$O(2^n)$** time in the worst case, and it

also carries stack depth proportional to the number of turns. Completely unusable past small toy arrays — a 100-stone array already puts you well beyond a reasonable time budget.

Complexity: Time $O(2^n)$, Space O(n) (recursion depth) plus whatever memoization you bolt on.

Optimal Solution

The optimal solution comes from two separate insights stacked on top of each other.

Insight 1 — prefix sums replace repeated summation. Every score a player can earn by cutting at index `i` is the sum of `stones[0..i]`. Computing that sum from scratch every time you consider a cut costs O(n), and if you do that inside a DP over every index, you're back to $O(n^2)$. Precompute the prefix sums once, in a single forward pass, so any prefix sum is a single array lookup:

```
prefix[i] = stones[0] + stones[1] + ... + stones[i]
```

Insight 2 — the DP itself has only one real transition, and it runs backward. Define `best[i]` as the best achievable score-gap for whichever player is "up" starting from index `i` (i.e., treating the game as if it begins with the stones `stones[i..n-1]`, with the current player able to cut anywhere from `i` to the end).

At index `i`, the current player has exactly two meaningful options, not "every possible split": - **Cut here:** take everything up to `i` as your prefix score, then subtract whatever the *opponent's* best outcome is starting from `i+1` (because from your perspective, their gain is your loss — this is the classic minimax "your score minus their optimal counter-score" trick). - **Don't cut here — let a later, better cut point stand:** carry forward `best[i+1]`.

So the recurrence is:

```
best[i] = max(prefix[i] - best[i+1], best[i+1])
```

Why does this not need to branch over every possible `x` like the brute force? Because the option "cut later instead of now" is already captured by `best[i+1]` — you don't need to separately consider cutting at `i+2`, `i+3`, etc. from position `i`, because whichever of those was better is already baked into `best[i+1]`.

You compute this **from right to left**, because `best[i]` depends on `best[i+1]` — the later position must already be solved before the earlier one can be. The last cut point (second-to-last stone) is forced — there's no choice left, since only one stone will remain after — so `best` starts initialized at the very last prefix sum, and the loop walks backward from there.

Index	Stone	Prefix sum	best[i] (computed right→left)
4	-5	-3	-3 (forced start)
3	4	1	max(1 - (-3), -3) = 4
2	-3	-2	max(-2 - 4, 4) = 4
1	2	0	max(0 - 4, 4) = 4
0	-1	-1	(not needed — index 0 isn't a valid cut)

The scan starts from the second-to-last index and stops at index 1, since a cut must take at least two stones (index 0 alone isn't a valid stopping point).

Python Code

```
from itertools import accumulate
from typing import List


def stoneGameVIII(stones: List[int]) -> int:
    """Return Alice's optimal score minus Bob's, both playing optimally."""
    prefix = list(accumulate(stones))
    n = len(prefix)

    # The last possible cut is forced: only one stone remains after it.
    best = prefix[-1]

    # Walk backward; each index only needs the result just ahead of it.
    for i in range(n - 2, 0, -1):
        best = max(best, prefix[i] - best)

    return best
```

Code Walkthrough

- `accumulate(stones)` builds the running prefix sums in one linear pass — this is the entire "insight 1" step, done in one line via the standard library instead of a hand-rolled loop.
- `best = prefix[-1]` seeds the DP at the last valid cut point. With only two stones left, the current player has no choice, so this value is forced, not chosen.
- The `for` loop runs from `n - 2` down to `1` (inclusive), skipping index `0` because a cut needs at least two stones, so index 0 can never be a valid stopping point on its own.
- Inside the loop, `prefix[i] - best` represents "cut here, take this prefix, and subtract the opponent's best continuation." `max(best, ...)` chooses between cutting now or deferring to a better cut already found further along.

- The final value of `best` after the loop is the answer — the optimal Alice-minus-Bob gap from the very start of the game.

Dry Run

Using `stones = [-1, 2, -3, 4, -5]`:

1. `prefix = [-1, 1, -2, 2, -3]`
2. `best = prefix[-1] = -3`
3. `i = 3`: `best = max(-3, prefix[3] - best) = max(-3, 2 - (-3)) = max(-3, 5) = 5`
4. `i = 2`: `best = max(5, prefix[2] - best) = max(5, -2 - 5) = max(5, -7) = 5`
5. `i = 1`: `best = max(5, prefix[1] - best) = max(5, 1 - 5) = max(5, -4) = 5`
6. Loop ends. Return `5`.

Matches the hand-traced answer from the Example section: `5`.

Complexity Analysis

- **Time: O(n).** Building the prefix array is one linear pass; the backward scan is another linear pass. Two passes, no nesting, so the total stays linear in the number of stones.
- **Space: O(n).** The prefix array holds `n` values. The scan itself only needs a single running variable, but the prefix array is unavoidable extra storage (you could shrink this to O(1) by folding the prefix computation into the same backward pass, but keeping them separate keeps the logic clear).

These bounds are correct because every stone is visited a constant number of times (once to build its prefix sum, once during the backward scan), and no step revisits or re-derives work already done — which is exactly what the brute force failed to guarantee.

Alternative Solutions

You can fuse the prefix-sum pass and the backward scan into a single reverse pass by accumulating the suffix sum as you go, shaving the code down slightly and dropping to O(1) extra space beyond the input. It's a reasonable micro-optimization once you already understand the two-pass version, but it trades a bit of readability for marginal savings — worth mentioning in an interview as a follow-up, not necessarily worth reaching for first.

Edge Cases

- **Minimum valid array (2 stones):** the loop body never executes; `best` is just the sum of both stones, which is correct since the only move is forced.
- **All negative values:** the max/min logic still holds — "optimal" doesn't mean "positive," it means "best available given both players play to their own advantage."
- **All positive values:** greedy "always take more" intuition might work here, but the algorithm doesn't rely on that assumption, which is exactly why it also works when values are mixed.
- **Large arrays (near 100,000 stones):** this is where brute force dies and the O(n) solution is required; make sure your prefix sums don't silently overflow in languages without Python's arbitrary-precision integers (not an issue in Python, but worth flagging in interviews).
- **Repeated/duplicate values:** no special handling needed — the algorithm only cares about sums, not distinctness.

Common Mistakes

1. **Trying to branch over every cut point `x` inside a recursive/DP formulation.** This reintroduces the exponential blowup the prefix-sum trick was designed to eliminate — the recurrence only needs `best[i+1]`, not every future index.
2. **Forgetting the backward direction.** Computing `best` left to right breaks the recurrence, since `best[i]` depends on a value (`best[i+1]`) that doesn't exist yet.
3. **Recomputing prefix sums inside the loop instead of precomputing them.** This silently degrades the algorithm from O(n) to $O(n^2)$.
4. **Always taking the current prefix instead of comparing against the running best.** Without the `max(best, prefix[i] - best)` comparison, the algorithm forces every cut to happen at the earliest chance, which is wrong whenever an early cut is worse than waiting.
5. **Off-by-one errors on the loop bounds.** Starting the scan at `n - 1` instead of `n - 2`, or not stopping before index 0, either double-counts the forced last move or allows an invalid single-stone cut.

Interview Questions

- Why must the backward scan start from the *second-to-last* index rather than the last?
- How would you prove the recurrence `best[i] = max(prefix[i] - best[i+1], best[i+1])` is correct via induction?
- What changes if a player is allowed to take exactly one stone per turn instead of at least two?
- How would you modify this to return the actual sequence of cut points, not just the final gap?

- Can this problem be solved with O(1) extra space? Walk through how you'd eliminate the prefix array.

Similar Problems

- **LeetCode 877 (Stone Game):** the original two-player stone game — good for building minimax/DP intuition before tackling the prefix-sum variant.
- **LeetCode 1140 (Stone Game II):** introduces a variable move-size constraint, a good next step in game-theory DP complexity.
- **LeetCode 1406 (Stone Game III):** closer in spirit — small fixed choice sets per turn, similar min/max recurrence structure.
- **LeetCode 1690 (Stone Game VII):** another prefix/suffix-sum-driven variant, useful for reinforcing the "reduce state to one index" pattern.

Key Takeaways

Stone Game VIII teaches a pattern that shows up constantly in DP and game-theory problems: the naive state space (full move history) is almost always larger than the *true* state space (whatever minimal information actually determines the rest of the game). Once you compress "which stones were taken" down to "one running prefix-sum comparison," the exponential game tree collapses into a single O(n) backward scan. That compression instinct — find the smallest sufficient state — is the real skill this problem is testing.

Watch the Video

For the full walkthrough, including a live benchmark comparing the brute force and optimal versions and two quiz rounds to lock in the intuition, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a series focused on building real algorithmic intuition — not just memorizing solutions — for coding interview problems, Python fundamentals, and core computer science concepts. Each video and article pairs together to walk through the reasoning an experienced engineer actually uses when they first see a problem.

Call To Action

If this walkthrough helped the prefix-sum trick click, give the video a like on YouTube and subscribe for more daily LeetCode breakdowns. For deeper dives and write-ups like this one delivered straight to your inbox, subscribe to the Substack. Drop a comment with the LeetCode problem you'd like covered next, and share this with anyone prepping for coding interviews.

Quick Review

Stone Game VIII: what pattern/tell signals this DP shape?

Optional-prefix-take game with running max on suffix — think 'suffix max over cumulative sums', not per-move brute force.

Time complexity of the optimized solution?

O(n) — single backward pass tracking a running best value, after O(n) prefix sums.

Space complexity of the optimized solution?

O(n) for prefix sums (or O(1) extra if you compute prefix sums in place).

Common bug when coding the backward suffix-max pass?

Forgetting to seed the running max with the last prefix sum before looping, causing an off-by-one on the final index.

Stone Game VIII vs classic interval Stone Game DP — key difference?

Classic game uses $O(n^2)$ interval DP on take-from-ends; this one collapses to 1D since a move consumes a prefix, not an end.

Interview follow-up: why do prefix sums remove the need for nested loops?

Each player's score for taking prefix i is fixed (prefix_sum[i] minus opponent's best future), so no re-summing per choice is needed.

Stone Game VIII — free Python cheat sheet